04 — Đánh giá trên TEST set

Chạy thứ 5 (sau notebook 01, 02, 03). Input: `faidset-processed` (test_en.csv, test_vi.csv) + 2 model đã train với R-Drop (`xlmroberta-rdrop`, `hybrid-rdrop`). Output: bảng Precision/Recall/F1 trên TEST set (tổng + theo từng ngôn ngữ) cho cả 2 model, dùng cho Section 5 — Đánh giá của báo cáo.

Lưu ý: notebook 02 và 03 chỉ báo cáo kết quả trên **val.csv** (dùng để chọn checkpoint tốt nhất qua các epoch). Notebook này đánh giá model đã chọn trên **test_en.csv + test_vi.csv** — dữ liệu hoàn toàn chưa được model nhìn thấy trong suốt quá trình huấn luyện.

```
!pip install transformers torch scikit-learn sentencepiece langdetect
underthesea -q

import pandas as pd
import numpy as np
import torch
import torch.nn.functional as F
from torch.utils.data import Dataset, DataLoader
from transformers import AutoTokenizer,
AutoModelForSequenceClassification
from sklearn.metrics import f1_score, precision_recall_fscore_support,
classification_report
from underthesea import word_tokenize
from langdetect import detect_langs, DetectorFactory,
LangDetectException
from tqdm import tqdm
import json

DetectorFactory.seed = 42
torch.manual_seed(42)
np.random.seed(42)

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f"Device: {device}")
```

```
Device: cuda
```

```
# ── Đường dẫn dữ liệu ──────────────────────────────────────────
TEST_EN_PATH = "/kaggle/input/datasets/cminhnguyndsdsds/faidset-
processed/test_en.csv"
TEST_VI_PATH = "/kaggle/input/datasets/cminhnguyndsdsds/faidset-
processed/test_vi.csv"

# ── Đường dẫn model ────────────────────────────────────────────
# Model 1: XLM-RoBERTa + R-Drop (từ notebook 02)
XLM_MODEL_DIR = "/kaggle/input/datasets/cminhnguyndsdsds/xlmroberta-
rdrop"
```

```
# Model 2: Hybrid – RoBERTa (EN) + PhoBERT (VI), cả 2 + R-Drop (từ notebook 03)
RO_MODEL_DIR = "/kaggle/input/datasets/minhbodoi/hybrid-rdrop/roberta"
PH_MODEL_DIR = "/kaggle/input/datasets/minhbodoi/hybrid-rdrop/phobert"

MAX_LEN = 128
BATCH_SIZE = 32
LANG_THRESHOLD = 0.8  # ngưỡng routing, giống notebook 03

test_en_df = pd.read_csv(TEST_EN_PATH, encoding="utf-8-sig")
test_vi_df = pd.read_csv(TEST_VI_PATH, encoding="utf-8-sig")
print(f"Test EN: {len(test_en_df):,} mẫu | Test VI: {len(test_vi_df):,} mẫu")
print(f"Test tổng: {len(test_en_df) + len(test_vi_df):,} mẫu")
```

```
Test EN: 62,621 mẫu | Test VI: 3,278 mẫu
Test tổng: 65,899 mẫu
```

Phần 1 — Load 3 model (XLM-R, RoBERTa, PhoBERT)

```
def load_model(model_dir):
    tok = AutoTokenizer.from_pretrained(model_dir)
    model = AutoModelForSequenceClassification.from_pretrained(model_dir).to(device)
    model.eval()
    return tok, model

print("Loading XLM-RoBERTa + R-Drop...")
tok_xlm, model_xlm = load_model(XLM_MODEL_DIR)

print("Loading RoBERTa (EN) + R-Drop...")
tok_ro, model_ro = load_model(RO_MODEL_DIR)

print("Loading PhoBERT (VI) + R-Drop...")
tok_ph, model_ph = load_model(PH_MODEL_DIR)

print("\nĐã load xong 3 model!")
```

```
Loading XLM-RoBERTa + R-Drop...
```

{"model_id":"749578a69a97489b876c1b63641d92e2","version_major":2,"version_minor":0}

```
Loading RoBERTa (EN) + R-Drop...
```

{"model_id":"47798b9b748a48f7b2da63c7f685c35a","version_major":2,"version_minor":0}

```
Loading PhoBERT (VI) + R-Drop...
```

{"model_id":"5cf3aa7fbbbf421180703ea4fb27775b","version_major":2,"version_minor":0}

```
Đã load xong 3 model!
```

Phần 2 — Đánh giá XLM-RoBERTa trên TEST set

XLM-RoBERTa xử lý cả EN và VI bằng cùng một model, batch qua DataLoader (không cần routing logic).

```
class TextDataset(Dataset):
    def __init__(self, df, tokenizer, max_len, segment=False):
        self.texts = df["text"].tolist()
        self.labels = df["label"].tolist()
        self.tok = tokenizer
        self.max_len = max_len
        self.segment = segment

    def __len__(self): return len(self.texts)

    def __getitem__(self, idx):
        text = self.texts[idx]
        if self.segment:
            text = word_tokenize(text, format="text")
        enc = self.tok(text, max_length=self.max_len,
                       padding="max_length", truncation=True,
return_tensors="pt")
        return {"input_ids": enc["input_ids"].squeeze(),
                "attention_mask": enc["attention_mask"].squeeze(),
                "label": torch.tensor(self.labels[idx],
dtype=torch.long)}


def predict_batch(model, tokenizer, df, segment=False,
batch_size=BATCH_SIZE):
    loader = DataLoader(TextDataset(df, tokenizer, MAX_LEN,
segment=segment), batch_size=batch_size)
    preds, trues = [], []
    with torch.no_grad():
        for batch in tqdm(loader, desc="Evaluating"):
            out = model(input_ids=batch["input_ids"].to(device),

attention_mask=batch["attention_mask"].to(device))
            preds.extend(out.logits.argmax(-1).cpu().numpy())
            trues.extend(batch["label"].numpy())
    return np.array(preds), np.array(trues)
```

```
print("Utilities ready")
```

```
Utilities ready
```

```
# Test EN
print("XLM-RoBERTa – Test EN...")
xlm_preds_en, xlm_trues_en = predict_batch(model_xlm, tok_xlm,
test_en_df)

# Test VI
print("XLM-RoBERTa – Test VI...")
xlm_preds_vi, xlm_trues_vi = predict_batch(model_xlm, tok_xlm,
test_vi_df)

# Gộp tổng
xlm_preds_all = np.concatenate([xlm_preds_en, xlm_preds_vi])
xlm_trues_all = np.concatenate([xlm_trues_en, xlm_trues_vi])

print("\n=== XLM-RoBERTa – TEST EN ===")
print(classification_report(xlm_trues_en, xlm_preds_en,
target_names=["Human", "AI"], digits=4))

print("\n=== XLM-RoBERTa – TEST VI ===")
print(classification_report(xlm_trues_vi, xlm_preds_vi,
target_names=["Human", "AI"], digits=4))

print("\n=== XLM-RoBERTa – TEST TỔNG (EN+VI) ===")
print(classification_report(xlm_trues_all, xlm_preds_all,
target_names=["Human", "AI"], digits=4))
```

```
XLM-RoBERTa – Test EN...

Evaluating: 100%|██████████| 1957/1957 [08:09<00:00,  4.00it/s]

XLM-RoBERTa – Test VI...

Evaluating: 100%|██████████| 103/103 [00:24<00:00,  4.15it/s]


=== XLM-RoBERTa – TEST EN ===
              precision    recall  f1-score   support

       Human     0.8981    0.6371    0.7454     30899
          AI     0.7245    0.9296    0.8143     31722

    accuracy                         0.7853     62621
   macro avg     0.8113    0.7834    0.7799     62621
weighted avg     0.8102    0.7853    0.7803     62621


=== XLM-RoBERTa – TEST VI ===
```

```
              precision    recall  f1-score   support

       Human     0.9990    0.9796    0.9892      1958
          AI     0.9705    0.9985    0.9843      1320

    accuracy                         0.9872      3278
   macro avg     0.9848    0.9890    0.9867      3278
weighted avg     0.9875    0.9872    0.9872      3278


=== XLM-RoBERTa — TEST TỔNG (EN+VI) ===
              precision    recall  f1-score   support

       Human     0.9062    0.6575    0.7621     32857
          AI     0.7325    0.9323    0.8204     33042

    accuracy                         0.7953     65899
   macro avg     0.8193    0.7949    0.7913     65899
weighted avg     0.8191    0.7953    0.7913     65899
```

Phần 3 — Đánh giá Hybrid trên TEST set

Hybrid dùng routing logic giống notebook 03: phát hiện ngôn ngữ bằng `langdetect`, hard routing nếu $P(\text{lang}) \geq 0.8$, soft voting nếu không chắc. Vì cần routing theo từng mẫu, không dùng DataLoader batch mà predict từng văn bản một (giống đúng cách notebook 03 đã đánh giá Hybrid trên val.csv).

```
def detect_language(text):
    try:
        res = detect_langs(text[:500])
        top = res[0]
        return (top.lang if top.lang in ["en", "vi"] else "other"),
top.prob
    except LangDetectException:
        return "other", 0.0


def predict_proba(text, model, tokenizer, segment=False):
    if segment:
        text = word_tokenize(text, format="text")
    enc = tokenizer(text, max_length=MAX_LEN, padding="max_length",
                    truncation=True, return_tensors="pt")
    with torch.no_grad():
        logits = model(input_ids=enc["input_ids"].to(device),

attention_mask=enc["attention_mask"].to(device)).logits
    return F.softmax(logits, dim=-1)[0][1].item()
```

```
def hybrid_predict(text):
    lang, prob = detect_language(text)
    if prob >= LANG_THRESHOLD:
        if lang == "en":
            p_ai = predict_proba(text, model_ro, tok_ro)
        elif lang == "vi":
            p_ai = predict_proba(text, model_ph, tok_ph, segment=True)
        else:
            p_ai = (predict_proba(text, model_ro, tok_ro) +
                    predict_proba(text, model_ph, tok_ph,
segment=True)) / 2
    else:
        p_ai = (predict_proba(text, model_ro, tok_ro) +
                predict_proba(text, model_ph, tok_ph, segment=True)) /
2
    return 1 if p_ai >= 0.5 else 0


print("Hybrid predict function ready")
```

```
Hybrid predict function ready
```

```
print("Evaluating Hybrid — Test EN...")
hybrid_preds_en = [hybrid_predict(t) for t in
tqdm(test_en_df["text"].tolist())]
hybrid_trues_en = test_en_df["label"].tolist()

print("Evaluating Hybrid — Test VI...")
hybrid_preds_vi = [hybrid_predict(t) for t in
tqdm(test_vi_df["text"].tolist())]
hybrid_trues_vi = test_vi_df["label"].tolist()

hybrid_preds_all = hybrid_preds_en + hybrid_preds_vi
hybrid_trues_all = hybrid_trues_en + hybrid_trues_vi

print("\n=== Hybrid — TEST EN ===")
print(classification_report(hybrid_trues_en, hybrid_preds_en,
target_names=["Human", "AI"], digits=4))

print("\n=== Hybrid — TEST VI ===")
print(classification_report(hybrid_trues_vi, hybrid_preds_vi,
target_names=["Human", "AI"], digits=4))

print("\n=== Hybrid — TEST TỔNG (EN+VI) ===")
print(classification_report(hybrid_trues_all, hybrid_preds_all,
target_names=["Human", "AI"], digits=4))
```

```
Evaluating Hybrid — Test EN...
```

```
100%|██████████| 62621/62621 [13:00<00:00, 80.28it/s]
```

```
Evaluating Hybrid — Test VI...

100%|██████████| 3278/3278 [00:56<00:00, 58.08it/s]


=== Hybrid — TEST EN ===
              precision    recall  f1-score   support

       Human     0.9268    0.5805    0.7139     30899
          AI     0.7004    0.9553    0.8083     31722

    accuracy                         0.7704     62621
   macro avg     0.8136    0.7679    0.7611     62621
weighted avg     0.8121    0.7704    0.7617     62621


=== Hybrid — TEST VI ===
              precision    recall  f1-score   support

       Human     0.9995    0.9811    0.9902      1958
          AI     0.9727    0.9992    0.9858      1320

    accuracy                         0.9884      3278
   macro avg     0.9861    0.9902    0.9880      3278
weighted avg     0.9887    0.9884    0.9884      3278


=== Hybrid — TEST TỔNG (EN+VI) ===
              precision    recall  f1-score   support

       Human     0.9334    0.6044    0.7337     32857
          AI     0.7087    0.9571    0.8144     33042

    accuracy                         0.7812     65899
   macro avg     0.8210    0.7807    0.7740     65899
weighted avg     0.8207    0.7812    0.7741     65899
```

Phần 4 — Phân phối chiến lược Routing (Hybrid)

Ghi lại mỗi mẫu được xử lý theo nhánh nào (hard-roberta / hard-phobert / soft-other / soft-bilingual) để báo cáo tỉ lệ routing, giống bảng `tab:routing_stats` trong báo cáo.

```python
def routing_strategy(text):
    lang, prob = detect_language(text)
    if prob >= LANG_THRESHOLD:
        if lang == "en": return "hard-roberta"
        if lang == "vi": return "hard-phobert"
        return "soft-other"
    return "soft-bilingual"
```

```
print("Đếm routing strategy trên toàn bộ TEST set...")
all_texts = test_en_df["text"].tolist() + test_vi_df["text"].tolist()
strategies = [routing_strategy(t) for t in tqdm(all_texts)]

from collections import Counter
strategy_counts = Counter(strategies)
total = len(strategies)
print("\n── Phân phối Routing Strategy ──")
for strat, count in strategy_counts.most_common():
    print(f"  {strat:16s}: {count:>7,} ({100*count/total:.2f}%)")
print(f"  {'Tổng':16s}: {total:>7,} (100.00%)")
```

```
Đếm routing strategy trên toàn bộ TEST set...

100%|██████████| 65899/65899 [02:17<00:00, 477.86it/s]


── Phân phối Routing Strategy ──
  hard-roberta    :  62,588 (94.98%)
  hard-phobert    :   3,278 (4.97%)
  soft-other      :      21 (0.03%)
  soft-bilingual  :      12 (0.02%)
  Tổng            :  65,899 (100.00%)
```

Phần 5 — Tổng hợp bảng so sánh 2 model

Tổng hợp macro F1 (Precision/Recall/F1) theo từng ngôn ngữ và tổng, dùng để điền trực tiếp vào các bảng `tab:results_overall`, `tab:results_en`, `tab:results_vi`, `tab:model_summary` trong báo cáo LaTeX.

```
def summarize(trues, preds, name):
    p, r, f1, support = precision_recall_fscore_support(trues, preds,
average=None, labels=[0, 1])
    macro_p, macro_r, macro_f1, _ =
precision_recall_fscore_support(trues, preds, average="macro")
    acc = (np.array(trues) == np.array(preds)).mean()
    return {
        "name": name,
        "human_p": p[0], "human_r": r[0], "human_f1": f1[0],
"human_support": support[0],
        "ai_p": p[1], "ai_r": r[1], "ai_f1": f1[1], "ai_support":
support[1],
        "accuracy": acc,
        "macro_p": macro_p, "macro_r": macro_r, "macro_f1": macro_f1,
    }
```

```
rows = [
    summarize(xlm_trues_all, xlm_preds_all, "XLM-RoBERTa — TỔNG"),
    summarize(xlm_trues_en,  xlm_preds_en,  "XLM-RoBERTa — EN"),
    summarize(xlm_trues_vi,  xlm_preds_vi,  "XLM-RoBERTa — VI"),
    summarize(hybrid_trues_all, hybrid_preds_all, "Hybrid — TỔNG"),
    summarize(hybrid_trues_en,  hybrid_preds_en,  "Hybrid — EN"),
    summarize(hybrid_trues_vi,  hybrid_preds_vi,  "Hybrid — VI"),
]

summary_df = pd.DataFrame(rows)
pd.set_option("display.float_format", lambda x: f"{x:.4f}")
print(summary_df.to_string(index=False))

summary_df.to_csv("test_evaluation_summary.csv", index=False)
print("\nĐã lưu: test_evaluation_summary.csv")
```

```
              name  human_p  human_r  human_f1  human_support   ai_p
ai_r  ai_f1  ai_support  accuracy  macro_p  macro_r  macro_f1
XLM-RoBERTa — TỔNG   0.9062   0.6575    0.7621          32857 0.7325
0.9323 0.8204       33042    0.7953   0.8193   0.7949    0.7913
  XLM-RoBERTa — EN   0.8981   0.6371    0.7454          30899 0.7245
0.9296 0.8143       31722    0.7853   0.8113   0.7834    0.7799
  XLM-RoBERTa — VI   0.9990   0.9796    0.9892           1958 0.9705
0.9985 0.9843        1320    0.9872   0.9848   0.9890    0.9867
     Hybrid — TỔNG   0.9334   0.6044    0.7337          32857 0.7087
0.9571 0.8144       33042    0.7812   0.8210   0.7807    0.7740
       Hybrid — EN   0.9268   0.5805    0.7139          30899 0.7004
0.9553 0.8083       31722    0.7704   0.8136   0.7679    0.7611
       Hybrid — VI   0.9995   0.9811    0.9902           1958 0.9727
0.9992 0.9858        1320    0.9884   0.9861   0.9902    0.9880

Đã lưu: test_evaluation_summary.csv
```

Phần 6 — Lưu kết quả chi tiết (JSON)

Lưu toàn bộ classification report dạng dict, kèm phân phối routing, để dùng lại khi viết báo cáo hoặc đối chiếu lại sau này.

```
results = {
    "test_sizes": {
        "test_en": len(test_en_df),
        "test_vi": len(test_vi_df),
        "test_total": len(test_en_df) + len(test_vi_df),
    },
    "xlm_roberta": {
        "en": classification_report(xlm_trues_en, xlm_preds_en,
target_names=["Human", "AI"], output_dict=True),
        "vi": classification_report(xlm_trues_vi, xlm_preds_vi,
```

```
target_names=["Human", "AI"], output_dict=True),
        "total": classification_report(xlm_trues_all, xlm_preds_all,
target_names=["Human", "AI"], output_dict=True),
    },
    "hybrid": {
        "en": classification_report(hybrid_trues_en, hybrid_preds_en,
target_names=["Human", "AI"], output_dict=True),
        "vi": classification_report(hybrid_trues_vi, hybrid_preds_vi,
target_names=["Human", "AI"], output_dict=True),
        "total": classification_report(hybrid_trues_all,
hybrid_preds_all, target_names=["Human", "AI"], output_dict=True),
    },
    "routing_distribution": dict(strategy_counts),
}

with open("test_evaluation_results.json", "w", encoding="utf-8") as f:
    json.dump(results, f, ensure_ascii=False, indent=2)

print("Đã lưu: test_evaluation_results.json")
print("\nDone! Dùng summary_df / results để điền vào Section 5 (Đánh
giá) của báo cáo.")
```

```
Đã lưu: test_evaluation_results.json

Done! Dùng summary_df / results để điền vào Section 5 (Đánh giá) của
báo cáo.
```